

分 类 号_____

学 号 M201672825

学校代码 10487

密 级_____

华中科技大学

硕士学位论文

面向深度学习应用的 GPU 存储优化研究

学位申请人：葛希

学 科 专 业：计算机软件与理论

指 导 教 师：廖小飞 教授

答 辩 日 期：2019 年 05 月 27 日

Thesis Submitted in Partial Fulfillment of the Requirements
for the Degree of Master of Engineering

Research on GPU Storage Optimization for Deep Learning Applications

Candidate : Xi Ge
Major : Computer Software and Theory
Supervisor : Professor Xiaofei Liao

Huazhong University of Science and Technology
Wuhan 430074, P.R.China
May, 2019

摘要

由于 GPU 强大的计算能力，其已经成为训练神经网络的首选设备。随着神经网络变得更宽更深，网络模型的准确性得到提升，但是训练这些神经网络需要的 GPU 内存空间也越来越大。有限的 GPU 内存容量成为训练更深更宽神经网络的主要障碍。为了解决 GPU 内存不足的问题，研究人员提出了交换数据到主存和重计算等方法，这些方法使得在前向传播过程中，GPU 内存中只保留部分层的输出结果。然而这些方法对层之间的数据依赖关系缺乏深入分析，且对瓶颈层缺乏训练支持。

针对这些问题，提出了充分利用重计算节省 GPU 内存开销的深度学习系统 MEDL，其基于重计算的 GPU 内存管理器分别对非线性和线性神经网络提供 GPU 内存优化方法。在非线性网络中，通过空间重用和活跃度分析取消将工具层的前向输出结果作为检查点的必要性，工具层的输出所占的 GPU 内存空间在前向传播中可以直接被释放，从而大量减少使用重计算方法训练非线性神经网络时的内存需求。在线性网络中，通过细粒度的双缓冲技术减少瓶颈层的内存开销，使更宽的神经网络能够在有限的 GPU 内存中进行训练。

实验结果表明，系统 MEDL 可以在有限的 GPU 内存中训练更深更宽的神经网络。在相同的实验环境下，与现有的深度学习系统相比，MEDL 的内存消耗平均降低了 27.5%，而且具有 10.9% 的性能提升。当不断增加训练的批次大小以至于传统的系统不能训练时，MEDL 仍能正常训练。

关键字：深度神经网络，GPU 内存管理，重计算，活跃度分析

Abstract

Massive computation parallelism of GPU makes it to be indispensable to train deep neural networks (DNN). As deeper and wider DNNs improve their accuracy, the memory footprint of their training becomes larger. However, the limited GPU DRAM capacity become the primary barrier to train deeper and wider network in GPU. In order to mitigate GPU memory capacity issue, the offload-prefetch and recomputation are proposed to reduce memory footprint for the training on a GPU. The offload-prefetch and recomputation can reduce the memory footprint further by trading less memory consumption with data transmission over slow PCIe bus and more computation overhead respectively. And only part of the layer's output to be retained in the GPU memory during forward propagation. However, these methods lack in-depth analysis of the data dependencies of different layers in backward computing, and lack support for the bottleneck layers.

To solve this problem, the deep learning system MEDL, which fully exploit the recalculation to save GPU memory overhead, is proposed to provide memory optimization methods for non-linear and linear networks. In non-linear networks, the memory overhead of the utility layer is reduced by spatial reuse and recomputation, and the necessity of using the forward output of the tool layer as a checkpoint is eliminated. The GPU memory space occupied by the output of the utility layer can be released directly, greatly reducing the memory requirements when training neural networks using recomputation. In a linear network, the fine-grained double buffering technique reduces the memory overhead of the bottleneck layer, enabling a wider neural network to be trained in limited GPU memory.

Experimental results show that the system MEDL can train deeper and wider neural networks in limited GPU memory. In the same experimental environment, MEDL's memory consumption was reduced by an average of 27.5% compared to the existing deep learning system, and with a performance improvement of 10.9%. When the batch size of the training is increased and existing deep learning system can't work, the MEDL can still work normally.

Keywords: Deep neural networks, GPU memory management, Recomputation, Liveness Analysis

目 录

摘 要.....	I
ABSTRACT.....	II
1 绪 论	
1.1 课题背景	(1)
1.2 课题研究动机.....	(2)
1.3 国内外研究现状.....	(5)
1.4 论文研究内容.....	(6)
1.5 文章框架结构.....	(7)
2 基于重计算的 GPU 内存优化设计	
2.1 重计算方法中检查点的设计.....	(8)
2.2 面向非线性神经网络的 GPU 内存优化设计.....	(11)
2.3 面向线性神经网络的 GPU 内存优化设计.....	(15)
2.4 本章小结	(17)
3 基于重计算的 GPU 内存管理器的实现	
3.1 GPU 内存管理器的关键技术.....	(18)
3.2 基于重计算的 GPU 内存优化策略.....	(22)
3.3 本章小结	(24)
4 系统测试与结果分析	
4.1 测试环境.....	(26)
4.2 结果分析	(27)
4.3 本章小结	(31)
5 总结与展望	
5.1 工作总结	(32)
5.2 未来展望	(32)

华 中 科 技 大 学 硕 士 学 位 论 文

致 谢.....	(34)
参考文献.....	(36)
附录 1 攻读学位期间被录用的会议论文	(40)
附录 2 攻读学位期间申请的国家发明专利	(41)

1 绪论

本章首先对深度神经网络进行介绍，阐述限制神经网络训练的 GPU 内存容量不足问题。本章接下来介绍 GPU 内存优化的相关研究现状，分析和讨论当前各种内存优化方法的优点和不足，并引出本文的主要研究内容，最后描述本文的基本组织结构。

1.1 课题背景

深度神经网络由于其优异的表现已经被应用到生活中的各个领域，比如计算机视觉，自然语言处理，语音识别等等。经典的神经网络模型一直在沿着宽度和深度的维度不断演进。在线性神经网络模型中，通过增加宽度和深度引入更多的参数，AlexNet^[1], vgg16^[2]等经典的线性神经网络能够有效地提升模型的表达能力。为了解决较深的线性网络带来的训练难度增加、性能下降等问题，ResNet^[3], GoogLeNet^[4], DenseNet^[5]和 Inception^[6, 7]系列等非线性神经网络采用恒等映射和深度融合等技术帮助深度神经网络进行训练。随着神经网络的不断变深变宽，训练神经网络模型时的计算量和训练过程中产生的中间结果都越来越多，对训练设备的计算能力和内存容量也提出了更高的要求。由于 GPU 强大的计算能力，其已经成为训练神经网络模型的首选设备。目前商用 GPU 中最大的内存容量为 32GB，而批次大小为 256 的神经网络 vgg16 在前向和后向传播中产生的中间数据有 58GB，批次大小为 128 的网络 inception v4^[8]的中间数据有 70.6GB，已经远远超过了 GPU 内存的容量。由于 GPU 内存容量的有限导致为了在单 GPU 卡上训练神经网络，只能采用较小规模的神经网络或者在多 GPU 卡上通过并行的方式进行训练。因此减少训练神经网络时的内存开销，将大大提高深度神经网络的可扩展性。

使用 GPU 训练深度神经网络时，GPU 内存中的数据可以分为 4 个部分：模型参数、特征图、梯度图、工作区。模型参数是训练过程中最核心的部分，模型参数在每轮迭代中都会进行更新。特征图是前向传播过程中产生的中间结果，梯度图是后向传播过程中产生的中间结果。特征图在产生之后需要被保存在 GPU 内存中，以便在反向传播中用于计算生成梯度图，梯度图中包括反向传播中的误差和参数的更新梯度值。而工作区是保存临时变量或矩阵的一个缓冲区。

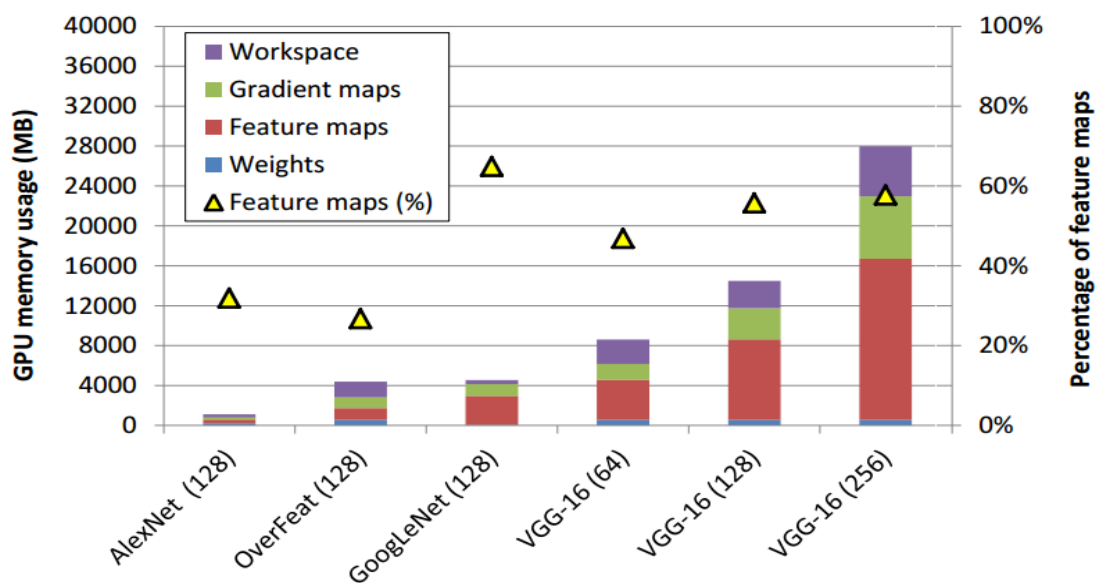


图 1.1 数据分布¹

如图 1.1 所示，在训练深度神经网络时，特征图的内存开销比重占到总内存开销的 50%以上。在训练过程中，绝大部分的内存都用于保存前向和后向传播过程中的中间结果。而参数的大小随着深度神经网络变得越深越宽，其在总的内存开销中的比例微乎其微。所以为了在有限的 GPU 内存中训练更深更宽的深度神经网络，首要的目标是降低前向传播中输出特征图的内存开销。

1.2 课题研究动机

目前流行的深度学习框架都为在 GPU 上进行神经网络训练提供了大量的支持，其中节省内存的主要方法包括活跃度分析，将数据交换到主存和重计算。其中活跃度分析方法在反向传播过程中直接丢弃不会再被使用的前向中间结果，使训练时的内存需求基本等同于前向传播过程中所有层的中间结果的总和。即使如此，批次大小等于 256 的 vgg16 网络在前向传播中产生的中间结果也有 39.4G，所以活跃度分析方法还是不能正常训练该情况下的 vgg16 网络。将数据交换到主存和重计算的方法可以更大程度的节省内存，他们分别以数据传输和重新计算前向中间结果的方式换取 GPU 中的内存空间，将数据交换到主存可以释放不参与当前层计算的数据所占的内存空间，重计算可以释放除检查点之外的所有数据的内存空间。随着 GPU 的不断更

¹ vDNN: Virtualized Deep Neural Networks for Scalable, Memory-Efficient Neural Network Design

新换代，GPU 的计算能力得到飞速提升，而通用的用于传输数据的 Pcie 带宽却提升缓慢，成为 GPU 加速训练的瓶颈，所以在训练较大较深的网络模型时应该首选重计算的方式恢复数据，避免出现大量数据传输严重拖慢训练速度的情况。因此如何降低重计算中检查点的 GPU 内存开销成为节省内存的关键。

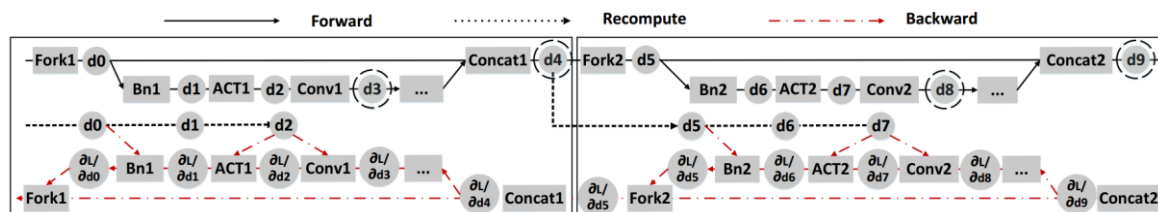


图 1.2 传统的重计算

深度神经网络分为线性网络和非线性网络，深度神经网络中包括大量不同类型的层。线性神经网络中每一层只包含一个前继层和一个后续层，而非线性网络中包含大量的非线性单元，每个非线性单元中存在大量的网络分支，即非线性单元中部分层可能包含多个前继层和多个后续层。

深度神经网络中通常包含各种不同类型的层，包括卷积层(CONV)，池化层(POOL)，激活层(ACT)，批规范化层(BN)，全连接层(FC)。传统的重计算方法丢弃计算成本较低的特征图，并保存计算耗时的结果，所以卷积层的输出特征图作为检查点被保存在 GPU 内存中，而池化层，激活层和批规范化层生成的中间结果在前向传播中直接被丢弃。在非线性神经网络中，除了存在大量的卷积层，池化层，激活层和批规范化层，还有大量的工具层。工具层对增加神经网络的非线性起到关键作用，其中 Fork 层和 Join 层, Concat 层最为常见。Fork 层在前向计算中共享其输入特征图到多个不同的后续层，增加神经网络的非线性。如图 1.2 所示，d4 和 d5 是同一个数据。在反向计算时，Fork 层对多个后续层的输入梯度图进行求和，从而获得 Fork 层输入数据的误差输入梯度图，然后将误差传播到上个非线性单元(Fork-Concat 块或者 Fork-Join 块)。Join 和 Concat 层的输入集合中都包含多个特征图，然而 Concat 层和 Join 层的不同点在于 Join 层是将多个同样大小的输入数据合并为一个，合并方法可以是相加、相乘、取最大值(最常用的方法是相加)，输出数据的大小与任意一个输入数据的大小相同；而 Concat 层是将多个任意大小的输入数据在某个维度上组合起来，输出数据的大小是所有输入数据大小的总和。在反向计算过程中，Join 层将输出数据的误差输出梯度图完整的复制给所有的输入梯度图，而 Concat 层将输出梯度图按照组合的维度分配给相应的输入梯度图。所以工具层在前向和反向传播中的计算开销都

很小而且其反向计算不需要使用其前向计算产生的中间结果。在传统的重计算设置中，如图 1.2 所示，除了将卷积层前向计算的输出设置为检查点，还需要将 Join 层和 Concat 层的前向计算的输出作为检查点，那么在反向传播中就可以恢复 Join 或 Concat 层到下一个非线性单元的所有分支中第一个卷积层之前的中间结果，如图 1.2 中的 d0, d1, d2 三个数据，然后反向计算就可以正常进行。虽然 Join 和 Concat 层产生的中间结果不参与 Join 和 Concat 层的反向计算，但是后续层的重新计算依赖这些中间结果。如果不将 Join 和 Concat 层的输出作为检查点，则需要对 Join 和 Concat 层的前向中间结果进行恢复，由于很多非线性网络的基本单元中存在大量短路连接，所以 Join 层和 Concat 层的中间结果只能从不存在于任何短路连接单元的最近距离的检查点开始恢复，这将产生大量的重计算开销和内存消耗，并不能达到节省内存的目的。而且通过统计发现，如图 1.3 所示，在传统的重计算设置中，工具层的内存占用在所有检查点中的比例可以高达 59%。所以如果在重计算方法中取消工具层的内存占用，将大大减少训练时总的内存开销。

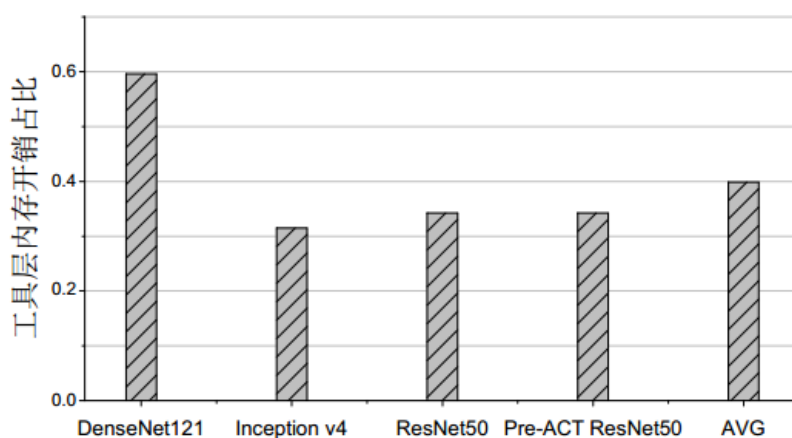


图 1.3 工具层的输出总大小在所有检查点中的比例

非线性神经网络在前向计算传播到非线性单元之前，往往会采用多次卷积和池化对输入特征图在不同维度进行快速降维，因此非线性网络的每个层在前向和后向传播中需要的内存比较小。但是非线性网络都是深度较深的网络，含有的层的数量巨大，因此在训练过程中同样需要大量的 GPU 内存。而线性网络正好相反，线性网络的深度一般较浅，比较经典的 VGG 网络也只有 16/19 层，但是靠近数据层的卷积层在前向和反向计算中一般需要较大的 GPU 内存，甚至超过 GPU 的容量。比如当批次大小为 256 时，VGG16 网络的最大层计算所需要的内存空间接近 12GB。所以解

决线性网络中瓶颈层的训练问题富有挑战。

1.3 国内外研究现状

训练神经网络的方式包括两种：数据并行和模型并行。模型并行为训练大规模的深度神经网络提供了直接的解决方案。**DistBelief**^[9]将神经网络划分成多份，每台 GPU 负责训练神经网络中的一部分。使用模型并行的方式进行训练时，分布式系统中的不同机器负责网络模型的不同部分。例如，神经网络模型的不同网络层被分配到不同的机器，或者同一层内部的不同参数被分配到不同机器。但是在训练过程中，不同机器之间会产生大量的通信开销，影响深度神经网络的训练速度。使用数据并行的方式进行训练时，不同的机器有同一个模型的多个副本，每个机器分配到不同的数据，然后将所有机器的计算结果按照某种方式合并。因此，数据并行的方式产生的通信开销较少，而且大量的深度学习系统使用数据并行的方式训练深度神经网络，因此本文同样关注以数据并行的方式进行训练时 GPU 内存容量不足的问题。

有限的 GPU 内存容量限制了 GPU 可以训练的神经网络的深度和宽度。内存不足问题一直受到深度学习从业者的关注，目前流行的深度学习框架基本都支持数据并行的方式进行训练，而且采用了各种不同的内存优化技术解决 GPU 内存容量不足的问题。**Caffe**^[10]和 **Torch**^[11]通过活跃度分析在反向传播中及时释放不再参与计算的前向输出数据的内存空间，从而将训练所需的内存开销减少到前向传播中所有中间结果的总和。**Mxnet**^[12]提出在前向传播中将计算密集型层的输出作为检查点，并在前向传播过程中只将检查点数据常驻 GPU 内存。在反向传播中，从检查点重新计算出非计算密集型层的输出，从而减少内存开销。**Tensorflow**^[13]的计算流程是基于其构建的有向无环图(DAG)，图中的节点表示各种运算，节点之间的连接表示张量数据的流动传输过程。**Tensorflow** 提议通过对有向无环图进行重用时间分析，将长时间未被重用的张量数据卸载到主存中，但是其缺乏 GPU 内存和主存之间的数据通信优化，严重影响神经网络的训练速度。

对于使用 GPU 训练神经网络，还有一些研究人员提出了更优的节省内存开销的方法。**vDNN**^[14]和 **Geeps**^[15]提出在主存和 GPU 内存之间交换数据的方法，以利用主存作为 GPU 的外部缓冲区。它们在反向传播过程中通过 CPU 和 GPU 之间的异步交换数据来隐藏部分通信开销。该方法的性能很大程度取决于通信和计算之间的比率，在大规模的神经网络中，一旦计算开销不足以隐藏通信开销，训练速度将迅速减慢。

Chen^[16]等人提出一种重计算的策略来节省内存开销，在前向传播中只保留检查点的数据，在反向传播中再从检查点通过重新计算的方式恢复其他层的前向中间结果。但是他们的方法在选择检查点时只考虑产生检查点数据的计算成本，并没有考虑层之间的数据依赖关系，从而没有充分利用节省内存开销的空间。SuperNeurons^[17]更进一步的使用统一内存池来支持数据的卸载和预取，并提供成本感知的重计算方法以减少内存消耗。Samuel^[18]等人通过考虑反向传播过程中各层之间的数据依赖性，将批规范化层和激活层融合到一起，提出了 In-Place Activated Batchnorm(InPlace-ABN)层。然而，该层的可逆性限制了激活层的选择范围，其融合的激活层必须具有可逆性。

除了使用软件的方法实现最先进的 GPU 内存管理，还有一些基于硬件的解决方案。Markthub^[19]等人通过直接访问 NVM 来打破 GPU 内存容量的限制，并使应用程序可以透明的使用驻留在 NVM 中的数据。还有一些研究提出专门用于深度神经网络的加速器^[20-25]，虽然这些定制的 ASIC 大大提高了训练神经网络的能效，但这些都没有解决训练神经网络时的内存容量瓶颈。

参数压缩同样可以减少内存开销。例如网络修剪^[26, 27, 28]的方法可以删除接近零值的参数，而量化^[29]和降低精度^[30]的方法使用低精度浮点数来节省内存。虽然减少参数在嵌入式系统上部署深度神经网络时具有极大的优势，但是参数在神经网络训练中只占据可忽略不计的内存使用量，所以参数压缩的方式在神经网络训练时并不会对减少内存开销起到显著的作用。

1.4 论文研究内容

根据研究背景和研究现状的介绍以及深度学习系统 GPU 内存优化中存在的挑战，引出本文的主要研究内容：通过对不同层之间在反向传播过程中的数据依赖关系，设计一种新的检查点设置策略，使重计算方法能够节省更多的 GPU 内存开销，同时为训练过程中的瓶颈层提供计算支持，从而使得更深更宽的神经网络可以在有限的 GPU 内存中进行训练。

本文的主要研究内容如下：

1. 通过对反向计算中数据依赖的深入分析，调整检查点的设置策略，减少了重计算中归一化操作的计算开销。
2. 在重计算的基础上，提出了面向非线性神经网络的 GPU 内存优化方法。非线性神经网络中存在大量的工具层，传统的重计算方法为了反向计算的顺利进行，将工

具层的输出特征图作为检查点常驻内存。通过空间重用和活跃度分析,使工具层在前向传播过程中不增加内存开销,降低训练过程中总的内存开销,使得更深的神经网络能够在有限的 GPU 内存中进行训练。

3. 在重计算的基础上,提出了面向线性神经网络的 GPU 内存优化方法。对于线性网络中的瓶颈层,本文使用更细粒度的双缓冲技术,支持层内重计算,减少瓶颈层在前向和后向传播中的内存开销,使得更宽的神经网络能够在有限的 GPU 内存中进行训练。

1.5 文章框架结构

本文的文章结构安排如下:

第一章首先结合课题的背景介绍传统 GPU 内存节省方法中存在的问题和挑战,引出本文的研究内容。然后是国内外减少训练过程中内存开销的研究现状,并介绍了本文的主要研究内容和组织结构。

第二章介绍了基于重计算的 GPU 内存优化设计。分别对线性神经网络和非线性神经网络进行深入分析,探索其训练过程中存在的内存节省空间,提出了面向线性和非线性网络的内存优化设计。

第三章详细介绍了基于重计算的 GPU 内存管理器的关键技术和优化策略实现。接着详细介绍了检查点设置方法和瓶颈层的训练流程。

第四章首先介绍了机器配置情况和实验环境等,然后对实验的数据集和测试的深度神经网络进行了说明,最后详细分析各个对比实验的结果。

第五章总结本论文的研究内容并对未来工作进行展望。

最后部分是致谢和参考文献。

2 基于重计算的 GPU 内存优化设计

本章描述三个优化技术:重计算方法中检查点的优化设计;面向非线性神经网络的 GPU 内存优化设计;面向线性的神经网络的 GPU 内存优化设计。第一个优化设计通过减少重计算的计算量提高系统性能,后两个优化可以减少训练过程所需要的最大内存,使得更大更深的神经网络能够进行训练。

2.1 重计算方法中检查点的设计

传统的重计算方法通常将卷积层的特征图作为检查点,然而并没有考虑层之间的数据依赖关系,本节提出将批规范化层的归一化结果作为检查点,减少重计算中的计算开销。同时将批规范化层的归一化结果作为检查点是设计 2.2 和设计 2.3 部分的基础。

cudnn 是通用的神经网络加速库,可以被集成到各种机器学习框架中。cudnn 为深度神经网络中常见的卷积层,批规范化层,激活层和池化层,以及非线性神经网络中的各种工具层提供优秀的算法实现。各种类型的层在前向计算和反向计算过程中所需要的数据如图 2.1 所示。

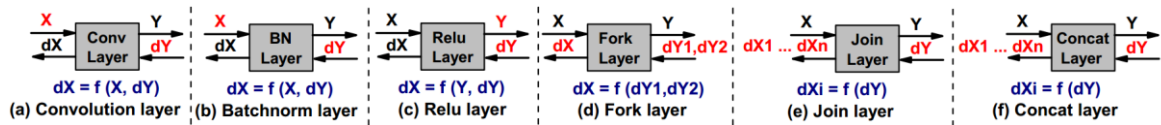


图 2.1 cudnn 中各层的数据依赖

批规范化层通过对数据分布进行归一化处理,从而使得网络的训练能够快速并简单,在一定程度上可以防止网络的过拟合。自从批规范化层出现以来,其已经成为深度神经网络中的一个重要组成部分。例如深度神经网络 ResNext 和 DPN 都在卷积层之后紧跟着一个批规范化层,而且深度神经网络 DenseNet 也在其卷积层之前默认插入批规范化层。

$$y'_i = \frac{x_i - \bar{x}}{\sqrt{\delta^2 + \epsilon}}, \quad \bar{x} = \frac{1}{m} \sum_{i=1}^m x_i, \quad \delta^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \bar{x})^2 \quad (2.1)$$

$$y = y' \cdot \gamma + \beta \quad (2.2)$$

$$\frac{\partial L}{\partial y'_i} = \frac{\partial L}{\partial y_i} \cdot \gamma, \quad \frac{\partial L}{\partial \gamma} = \sum_{i=1}^m \frac{\partial L}{\partial y_i} \cdot y'_i, \quad \frac{\partial L}{\partial \beta} = \sum_{i=1}^m \frac{\partial L}{\partial y_i} \quad (2.3)$$

$$\frac{\partial L}{\partial x_i} = (\delta^2 + \epsilon)^{-1/2} \left(\frac{\partial L}{\partial y'_i} - \frac{1}{m} \sum_{j=1}^m \frac{\partial L}{\partial y'_j} - \frac{1}{m} y'_i \sum_{j=1}^m \frac{\partial L}{\partial y'_j} \cdot y'_j \right) \quad (2.4)$$

批规范化层的前向计算过程分为两个部分归一化和尺度变换和偏移，其前向计算过程分别如公式 2.1 和 2.2 所示。归一化过程首先计算出该批次数据的均值 $\bar{x}$ 和方差 δ^2 ，接下来就是对输入特征图 $X(X = x_1, x_2 \dots x_m)$ 进行归一化操作，然后才能得到归一化后的值 y' 。尺度变换和偏移的计算过程中要引入平移参数 β 和缩放参数 γ ，利用缩放参数和平移参数对归一化后的数据 y' 进行处理得到最终的结果 y 。批规范化层的反向计算如图 2.1 中的(b)所示，在 cudnn 的实现中，批规范化层反向计算函数需要的参数为输入特征图 X 和输入梯度图 dY 。批规范化层反向计算中最终产生的数据为输出梯度图 dX 和缩放参数的梯度 $d\gamma$ ，平移参数的梯度 $d\beta$ ，从公式 2.3 和公式 2.4 可以看出批规范化层的反向计算真正需要的参数是归一化后的结果 y' 和输入梯度图 dY ，而输入特征图 X 并不真正被需要。从图 2.1 中的(a)中可以看出，卷积层的反向计算需要的数据是输入特征图 X 和输入特征图 dY 。而其输出特征图 Y 在卷积层的反向计算中并不起任何作用，在重计算方式中，其唯一的作用就是作为后续层重新前向计算的起点。

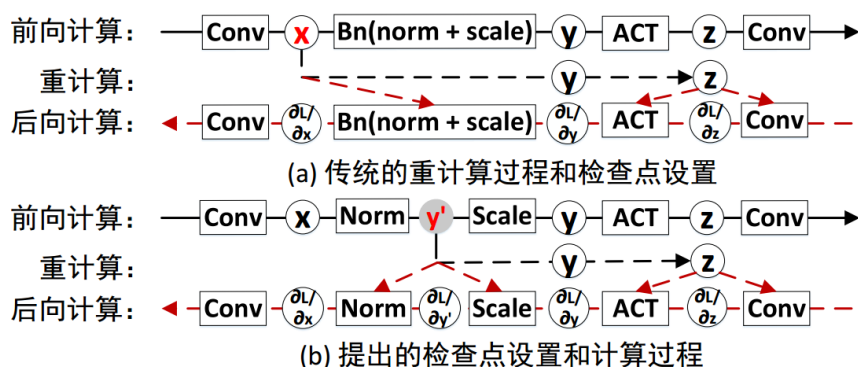


图 2.2 批规范化层的分解

在非线性神经网络中，残差单元由于解决了网络深度带来的退化问题而被广泛使用，而且残差网络更容易优化，还能够通过增加相当的深度来提高准确率。残差单元(residual unit)因为其良好的表现而被广泛用于各种非线性神经网络模型中。残差单元除了短路连接外还包括其他网络分支，网络分支的组成以卷积层，批规范化层，激活层为主。由于非线性网络的深度较深，在训练过程中需要的内存空间通常比较大，甚至超过 GPU 的内存容量。为了保证模型能够正常训练，以前的工作通常会采用重计算或者交换数据到主存的方式减少内存开销。传统重计算一般认为卷积层是计算

耗时的，所以将卷积层的输出特征图设置为检查点，在前向传播过程中将卷积层的输出特征图一直保存在 GPU 内存中，而其他层的输出特征图在前向传播中直接被丢弃，在反向传播中再从最近的检查点通过重新的前向计算进行恢复。传统的交换数据到主存的方法为了更好的隐藏通信开销，通常首先将除卷积层的输出特征图之外的数据卸载到主存中。总而言之，无论采用哪种方式，都是尽可能多的保留了卷积层的输出特征图在 GPU 内存中。在非线性神经网络的分支中，卷积层通常和批规范化层成对出现，卷积层的后续层也通常是批规范化层，那么常驻内存的卷积层的输出特征图同样是批规范化层的输入特征图。所以卷积层的特征图并不真正参与任何反向计算，只是作为其他层重计算的起点。此时在内存中保存卷积层的输出特征图 Y 和保留批规范化层的中间结果 y' 对内存的需要都是基本相同的，但是能够节省重计算中数据归一化的计算开销。

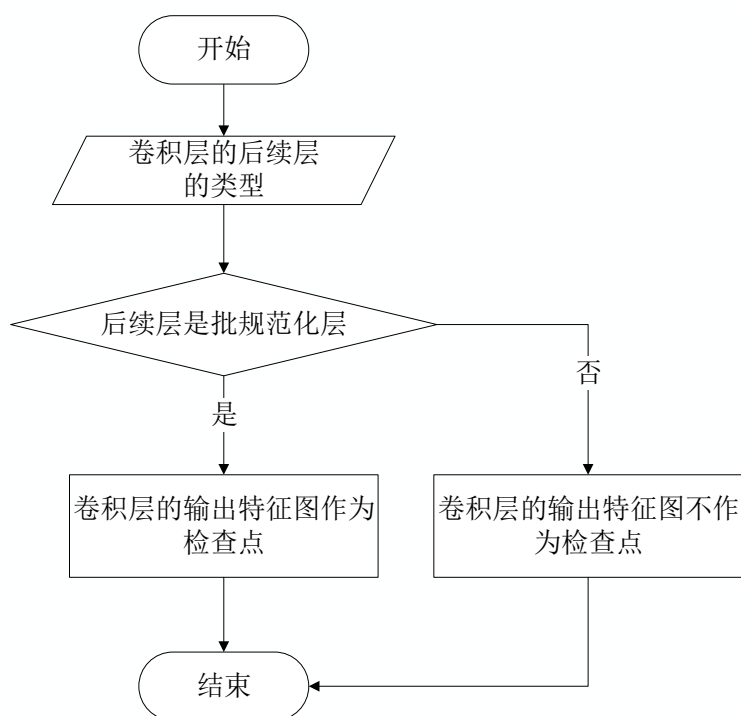


图 2.3 卷积层输出特征图的检查点设计

针对上文中提到的现象，本节对检查点的设计进行优化，首先如图 2.2 所示，将批规范化层分解为 Norm 层和 Scale 层，Norm 层的输出特征图则对应于批规范化层的归一化结果 y' ，所以将 Norm 层的输出特征图为检查点，常驻在 GPU 内存中。卷积层的输出特征图的检查点设置如图 3.3 所示，具体包括：

1. 如果卷积层的后续层为批规范化层,则卷积层的输出特征图既不参与任何的反向计算,又不作为后续层的重计算的起始点。所以此时卷积层的输出特征图不作为检查点,在前向传播中可以直接丢弃。
2. 如果卷积层的后续层不是批规范化层,则其输出特征图仍然作为后续层重计算的起始点,所以此时卷积层的输出特征图作为检查点,常驻 GPU 内存中,用于在重计算中恢复后续层的中间结果。

2.2 面向非线性神经网络的 GPU 内存优化设计

非线性神经网络的深度一般较深。非线性神经网络包含大量的非线性单元(残差单元等),每个非线性单元都是一个 Fork-Join 块或者 Fork-Concat 块。本文将非线性单元的形式总结为图 2.4 中的四种形式。形式 1 最初出现在深度神经网络 ResNet 中,得益于短路连接的恒等映射技术,形式 1 可以极大的增加神经网络的深度。形式 2 最早出现在 pre-act ResNet 中,其采用恒等的短路连接和预先激活的恒等映射技术,从而可以保证梯度在反向传播中的光滑性以及一定程度上缓解梯度消失问题。形式 1 和形式 2 的不同之处在于卷积层,批规范化层和激活层的组合顺序。在 Fork-Concat 块中,如果存在短路连接,那么批规范化层(BN),卷积层(CONV)和激活层(ACT)一般采用 BN-ACT-CONV 的组合顺序,如深度神经网络 DenseNet 中存在大量形式 3 的非线性单元。如果非线性单元的分支中不存在短路连接,如在神经网络 ResNext 和 Inception 系列中,则采用大量形式 4 的非线性单元。在同一个神经网络中也可能包含多种形式的非线性单元,比如在深度神经网络 DPN 中形式 1 和形式 3 就同时存在,神经网络 ShuffleNet 也同时包含形式 1 和形式 4 的非线性单元。这四种形式基本上包含了目前所有的经典非线性网络的非线性单元的结构,每种形式中由于采用不同类型的工具层(Join/Concat 层)和批规范化层,卷积层,激活层的不同组合顺序,存在不同的内存节省空间。下文的设计部分以这四种形式的非线性单元为例分析非线性神经网络种的内存节省空间。图 3.4 表示四种形式,其中包括三个计算过程:前向计算过程,重计算过程和反向计算过程,用虚线圈住的圆圈表示检查点数据,红色的圆圈表示空间复用。

上一章中提到传统的重计算方法中,由于非线性神经网络中存在大量的短路连接,所以在设计重计算的检查点时,必须将 Join 层和 Concat 层的输出特征图作为检查点。那么 Join 层和 Concat 层是否还存在内存节省的空间呢?而且上节中提到将批

规范化层分解为 Norm 层和 Scale 层的形式，并将 Norm 层的输出特征图设置为检查点，该输出特征图被保留在 GPU 内存中。通过分析发现将 Norm 层的输出特征图作为检查点为 Join 层和 Concat 层的内存节省提供了设计空间，而且非线性单元中批规范化层和卷积层不同的先后顺序对内存节省设计产生不同的影响。具体分析如下所示。

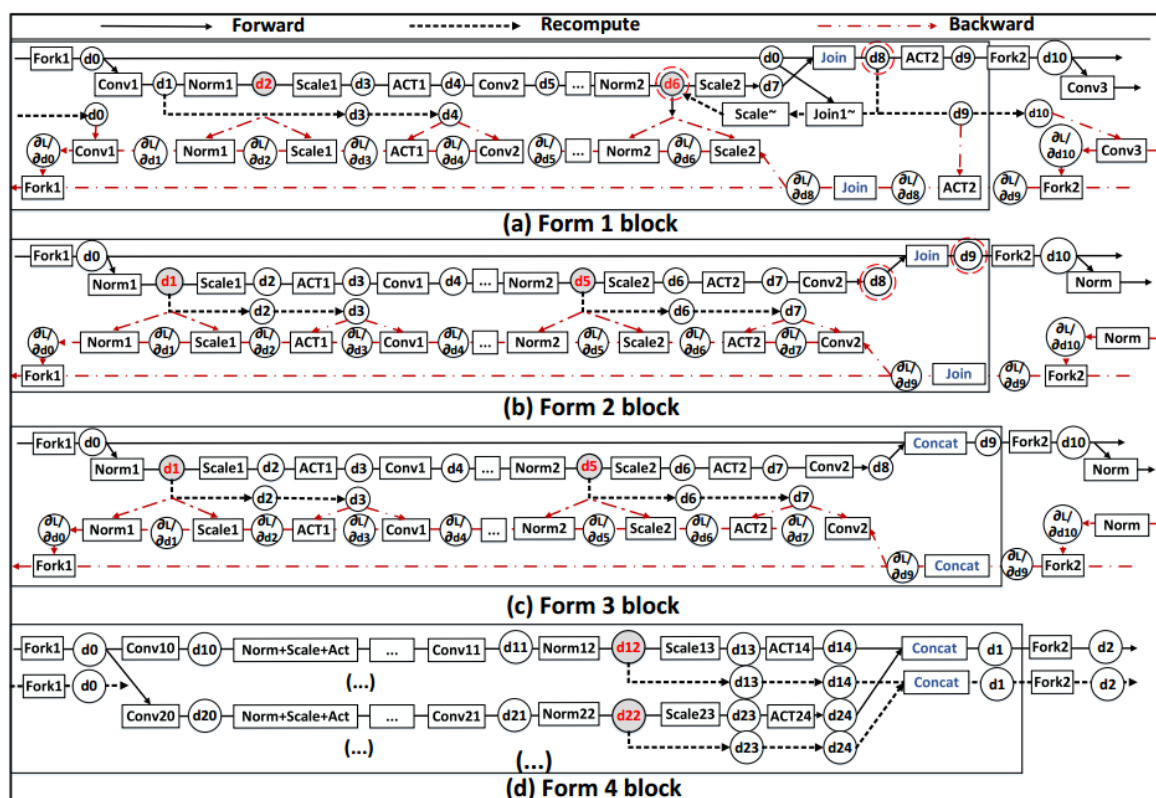


图 2.4 非线性单元的各种形式

1. CONV-BN-ACT 类型：如图 2.4 中的(a)和(d)所示，卷积层，批规范化层（Norm 层和 Scale 层的组合），激活层的组合顺序都为 CONV-BN-ACT。

图 2.4 (a) 中，在非线性单元（Fork-Join 块）内部，不和 Fork 层临近的卷积层的输入特征图可以通过从 Norm 层的输出特征图开始计算得到，如 d5 可以从 d2 开始计算得到，而和 Fork 层临近的卷积层的输入特征图 d0 或 d10 只能从上一个非线性单元中最后一个检查点重计算得到。按照传统的重计算方法，Join 层的输出特征图作为检查点，则 Join 层的输出特征图 d8 需要常驻内存，除此之外由于 Norm 层的输出特征图为检查点，所以其输出数据 d6 也需要常驻内存。假设上一个非线性单元的 Join 层的输出特征图为 d8',那么 Join 层一个输入特征图 $d0 = \text{act}(d8')$ ，那么 Join 层的另外一个输入特征图只需要从数据 d6 处通过一个 Scale 层的重计算就可以得到。有

了 Join 层的输入特征图,就能轻易地计算出 Join 层的输出特征图,即 $d8 = \text{scale}(d6) + d0 = \text{scale}(d6) + \text{act}(d8')$, 而且 Scale 层和激活层的计算开销都很小,所以有任意两个检查点的数据就可以得到另外一个检查点的数据,检查点的数据存在冗余,所以可以利用少量的重计算开销减少 Join 层的内存消耗。(a) 所代表的非线性单元存在短路连接,所以 Join 层的输出特征图必须常驻内存,而且 Join 层的所有输入和输出特征图的空间大小相等,即 $\text{size}(d6) = \text{size}(d8)$, 数据 d6 由于是 Norm 层的输出特征图,所以 d6 是检查点,在前向传播中常驻内存。因此在前向传播中可以重用数据 d6 的内存空间保存 Join 层的输出特征图 d8。在反向传播到 Join 层之前,可以直接从数据 d8 开始进行重计算,恢复 Join 层的所有后续层的中间结果(d9, d10),在反向计算到 Join 层时通过矩阵相减的方式得到 $\text{scale}(d6) = d8 - d0$, 然后通过 Scale 层的逆计算得到原始的 b6, 整个过程中只需要额外的 Join 层和 Scale 层的逆计算开销。Join 层由于和临近的上一个检查点(此处为 Norm 层的输出特征图)共享内存空间,所以 Join 层并不会增加内存消耗。其他的数据如 d1 和 d5 在反向传播中不起作用,激活层和卷积层的反向计算所需要的数据 d3, d4 可以从数据 d2 开始重计算得到。所以通过 Join 层的空间复用,包含大量形式 1 的非新型单元的的网络的前向传播中,只需要保存所有 Norm 层的输出特征图,在反向传播的时候就可以重计算出其他所有层的前向中间结果,保证反向计算正常进行。

图 2.4 (d)中,在非线性单元(Fork-Concat 块)内,Concat 层对所有的输入特征图进行某个维度的组合,所以 Concat 层的输出特征图的大小是所有输入特征图大小的总和,即 $\text{size}(d1) = \text{size}(d14 + d24 + \dots)$, 而且 Concat 层的输出特征图不参与该层的反向计算。将 Concat 层的输出特征图作为检查点,那么 Concat 层的输出特征图常驻内存的作用只是作为重计算的起始点。由于 Concat 层的输出特征图大小在整个非线性单元中占比较高,节省 Concat 层的内存开销将大量减少整个网络在训练时的内存需求。而且由于形式 4 代表的非线性单元中不存在短路连接,形式 4 表示的非线性单元中的每个分支中都存在检查点,通过这些检查点可以重新计算出 Concat 的输出特征图。所以将 Concat 层的输出特征图作为检查点会产生数据冗余。由于 Norm 层的输出作为检查点,其输出数据要求常驻内存,所以可以从与 Concat 层相邻的 Norm 层的检查点开始重计算(从 d12, d22 等开始重计算,恢复数据直到 d2),以少量的 Scale 层和激活层的计算开销为代价避免 Concat 层对内存的消耗。

在 CONV-BN-ACT 组合中,Join 层的输出特征图重用临近的检查点的内存空间,

而临近的检查点为 Norm 层的输出特征图, 该特征图参与 Norm 层的反向计算, 并作为后续层重计算的起点, 所以 Norm 层的输出特征图常驻内存, Join 层的输出特征图也常驻内存, 作为后续层重计算的起点。对于 Concat 层的输出特征图, 则直接取消将其作为检查点。因为 Join 层重用临近的检查点的内存空间, 所以 Join 层和 Concat 层在前向传播过程中并没有内存开销, 且非线性单元在前向传播中都只需要保存 Norm 层的输出特征图在 GPU 内存中, 其他所有层的中间结果直接被丢弃。

2. BN-ACT-CONV 类型: 如图 2.4 中的(b)和(c)所示, 卷积层, 批规范化层 (Norm 层和 Scale 层的组合), 激活层的组合顺序都为 BN-ACT-CONV。在前向传播过程中同样可以只保存 Norm 层的输出特征图作为检查点。以图 2.4 (b)为例, 因为 Norm 层的反向计算并不需要输入数据, 所以在反向计算中, 不同的非线性单元之间不存在数据依赖, Join 层的输出 d9 既不用于该层的反向计算, 又不用于重计算恢复后续层的输出数据。激活层和卷积层反向计算需要的是同一个参数, d3 或 d7, 而且他们都可以保留在内存中的 Norm 层的中间结果 d1 或者 d5 重计算得到, 所以在非线性单元中只保留数据 d1 和 d5 即可保证整个非线性单元反向计算的正常进行。上文对 CONV-BN-ACT 组合的设计中, 为了避免 Join 层的内存开销, 使 Join 的输出特征图重用距离最近的检查点的数据空间。图 2.4 (b) 中离 Join 层最临近的检查点为卷积层的输出特征图, 因为卷积层的下一层是工具层, 所以该卷积层的输出特征图 d8 是检查点。由于数据 d8 和 d9 并不参与任何反向计算, 所以 Join 层的输出特征图 d9 可以安全的重用检查点 d8 的内存空间。为了最大程度的节省内存, 检查点 d8 可以通过活跃度分析在前向传播中直接丢弃。因此, 在前向传播中, 形式 2 的非线性单元同样只需要保留 Norm 层的输出特征图, 将其作为检查点。同理存在于图 2.4 (c)所示的非线性单元中, 可以直接取消对 Concat 层的输出特征图的检查点设置。检查点 d8 由于及不参与卷积层 Conv2 的反向计算, 又不参与 Concat 层的反向计算, 所以可以通过活跃度分析在前向传播中直接释放数据 d8。

在 BN-ACT-CONV 组合中, Join 层的输出特征图仍然重用临近的检查点的内存空间, 重用的是卷积层的输出数据空间, 该数据既不参与卷积层的反向计算又不用于恢复后续 Join 层的输出特征图, 所以可以通过活跃度分析在前向传播中直接被释放该数据。对于 Concat 层, 仍然可以取消将 Concat 层的输出特征图作为检查点。由于 Join 层重用临近的检查点的内存空间, 所以 Join 层和 Concat 层在前向传播过程中并没有内存开销, 所以非线性单元在前向传播中仍然只需要保存 Norm 层的输出特征

图在 GPU 内存中，其他所有层的中间结果直接被丢弃。

通过对非线性网络中工具层的检查点的深入分析，以及融合活跃度分析到重计算中，本节提出面向非线性神经网络的内存管理策略。由于该方法基于重计算，所以重点在于减少常驻内存的检查点的总量。首先对于 Concat 层可以直接取消将其输出特征图作为检查点。对于 Join 层，其输出特征图重用临近的检查点的内存空间，所以 Join 层和 Concat 层在前向传播过程中并不产生内存开销。然后通过活跃度分析及及时释放不参与后续计算的检查点。通过这种方式，在非线性深度神经网络的前向传播中，只保存 Norm 层的输出特征图在 GPU 内存中，减少传统的重计算设计中需要常驻内存的数据总量，使得在有限的 GPU 内存中可以训练更深的神经网络模型。

2.3 面向线性神经网络的 GPU 内存优化设计

线性网络的深度一般较浅，但是每层计算所需的内存空间较大，所以较宽。比较经典的 VGG 也只有 16/19 层，但是在靠近输入层的卷积层在前向和反向计算中一般需要较大的内存。为了在有限的内存中训练较大的网络，以前的工作通常采用交换数据到主存的方式减少内存开销。在前向传播中，中间结果被卸载到主存中，在反向传播被使用之前，再从主存中预取回 GPU 内存。此时内存最大占用为神经网络中最大层的所有特征图和梯度图的总和，如果最大层的内存占用超过 GPU 内存容量，训练就不能正常进行。

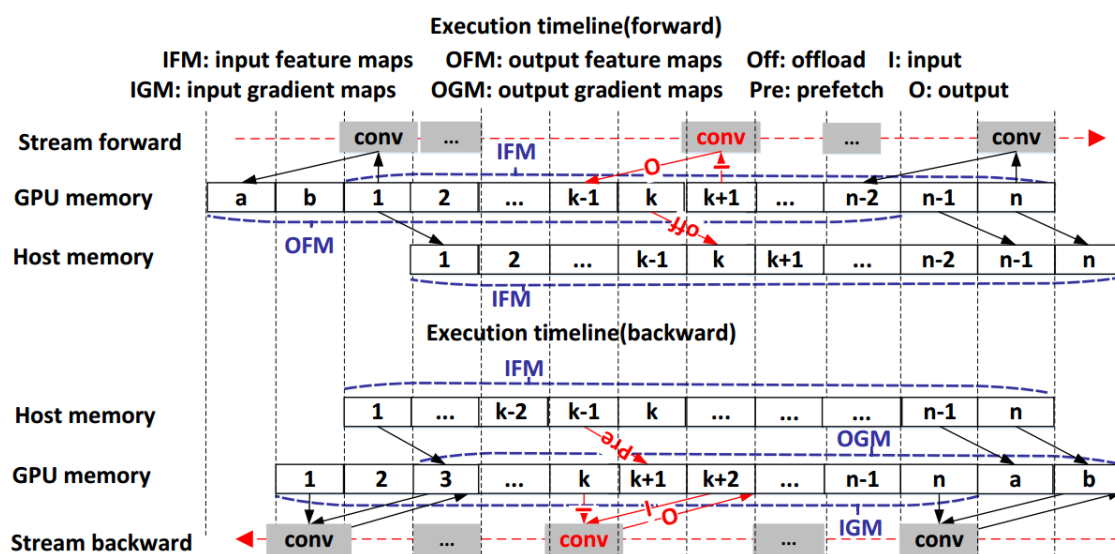


图 2.5 双缓冲方法详细设计

$$\frac{M \cdot (n+2)}{n} + \text{size}(\text{conv}(\frac{M}{n})) \leq N \quad (2.5)$$

传统的重计算被用于后向传播,用以恢复前向传播中的中间结果。为了解决线性网络中的瓶颈问题,本节提出细粒度的双缓冲方法,支持层内的重计算。线性网络的瓶颈层通常是靠近数据层的卷积层,如图 2.5 所示,将卷积层的输入特征图按照批次的维度划分为 n 个 block,除此之外,还需要申请两个额外的缓冲区 a 和 b 。 a 和 b 的大小与每个 block 的大小相同。 n 的取值为满足公式 2.5 的最小正整数,其中 M 表示该卷积层原始的输入特征图大小, $\text{size}(\text{conv}(M/n))$ 表示每个 block 在卷积计算所需要的内存空间, N 表示 GPU 容量。不同的 block 分别进行卷积时,同一时刻只有一个 block 的数据用于卷积计算,每个 block 在前向和反向计算中也只会被使用一次,所以每个 block 中的数据空间在大部分时间处于闲置。为了保证该卷积层的正向和反向传播,本设计最大程度的重用输入特征图和输出特征图的地址空间。当 n 满足公式 2.5 时,按照如下步骤进行计算:

1. 前向计算: 从第一个 block 开始计算,第一个 block 产生的输出结果保存在缓冲区 a 中,第二个 block 的输出结果保存在缓冲区 b 中。在进行第二个 block 的卷积时,将第一个 block 的输入数据卸载到主存中,在第二个 block 的计算和第一个 block 的数据传输这两个步骤都完成后,才能继续下一个 block 的卷积计算,通过这种方法可以隐藏一部分通信开销。同理,在进行到第 k 个 block 的卷积计算的同时,卸载第 $(k-1)$ 个 block 中的输入数据到主存中,并保存输出结果在第 $(k-2)$ 个 block 中。当所有的 block 都完成卷积计算后,缓冲区 a , b 和输入特征图中前 $(n-2)$ 个 block 的数据一起作为下一层的输入。该卷积层完成前向计算后可以直接丢弃其输入数据,因为主存中已经保存了该输入数据副本。

2. 反向计算: 上述提到的方法同样可以应用到反向传播过程。将 GPU 内存中的输出梯度图划分为 n 个 block,同样分配额外的两个缓冲区 a 和 b 。在反向计算应用到第 k 个 block 之前,其相应的输入特征图已经被预取到第 $(k+2)$ 个 block 中,反向计算产生的输出梯度图同样被保存在第 $(k+2)$ 个 block 中。在第 k 个 block 的反向计算进行的同时,预取主存中输入特征图的第 $(k-1)$ 个 block 数据到 GPU 内存中输入梯度图的第 $(k+1)$ 个 block 中。

通过这种方式,可以尽可能多的重叠计算和通信开销,从而减少 GPU 的停顿周期。同时,通过细粒度的双缓冲重用输入和输出数据空间,减少瓶颈层的内存开销。不同 block 在前向计算和后向计算中共享相同的参数,所以不同 block 之间的计算操作完全相同,所以多个 block 的计算可以视为层内的重计算,参数梯度在不同 block

的后向计算中被累积。

2.4 本章小结

本章首先介绍了重计算中面向卷积层的检查点设置方法的缺陷，提出了新的节省计算开销的检查点设置方法。接着对非线性网络进行 GPU 内存优化分析，进一步对非线性单元中工具层的检查点设置进行优化，通过空间重用和活跃度分析减少常驻内存的检查点的总量，有效的减少了训练过程中总的内存开销。然后介绍了面向线性神经网络的 GPU 内存优化分析，提出了一种更细粒度的双缓冲计算方法，通过层内的重计算，降低瓶颈层的内存开销，有效的解决了瓶颈层的计算问题。

3 基于重计算的 GPU 内存管理器的实现

在系统 SuperNeurons 上实现上文提出的基于重计算的内存优化设计(MEDL), MEDL 的实现借助于 SuperNeurons 系统提供的如图 3.1 所示的活跃度分析模块(liveness)、交换数据到主存模块(lru)、重计算模块(recompute)和内存池模块(unified tensor pool)。MEDL 的重计算模块的 API 对用户是透明的, 用户只需要关注神经网络的设计, 不用考虑 GPU 内存短缺问题。

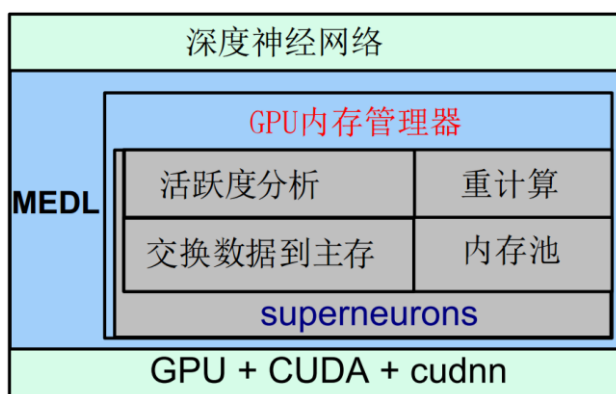


图 3.1 MEDL 系统架构

3.1 GPU 内存管理器的关键技术

本节主要介绍训练神经网络时的四种内存优化技术。为了便于分析这些技术对内存开销的影响, 本文将第 i 层在前向传播中的内存开销表示为 l_i^f , 在反向传播中的内存开销表示为 l_i^b 。而且将训练过程中内存开销的峰值表示为 $peak_m$ 。传统的内存分配策略在训练神经网络时, 为网络中的每一个数据单独分配内存空间, 本节将传统的面向整个网络的内存分配策略作为基准, 所以传统的内存分配策略的峰值 $peak_m = \sum_{i=1}^N l_i^f + \sum_{i=1}^N l_i^b$, 其中 N 表示深度神经网络的层数。

3.1.1 活跃度分析

训练深度神经网络时, 由于反向传播的计算需要使用前向传播产生的中间结果, 所以前向传播中产生的中间结果必须常驻内存。但是所有中间结果的生命周期并不完全相同。由于反向传播和前向传播的计算顺序正好相反, 所以越靠近输入层的中间结果, 其生命周期越长; 越靠近输出层的中间结果, 其生命周期越短。因此活跃度分析以中间结果的生命周期为依据, 使不同的数据在不同的时间段重用相同的物理

内存空间。活跃度分析的流程一般包括两个步骤：首先，为神经网络中的每一层分别构建一个输入和输出集合，然后运行时通过检查输入集合中的数据和后续层的依赖关系来填充输出集合。如果输入集合中的数据在后续层的计算中都不再被需要，那么该数据将不被填充到输出集合，而且该数据的内存空间可以直接被释放。

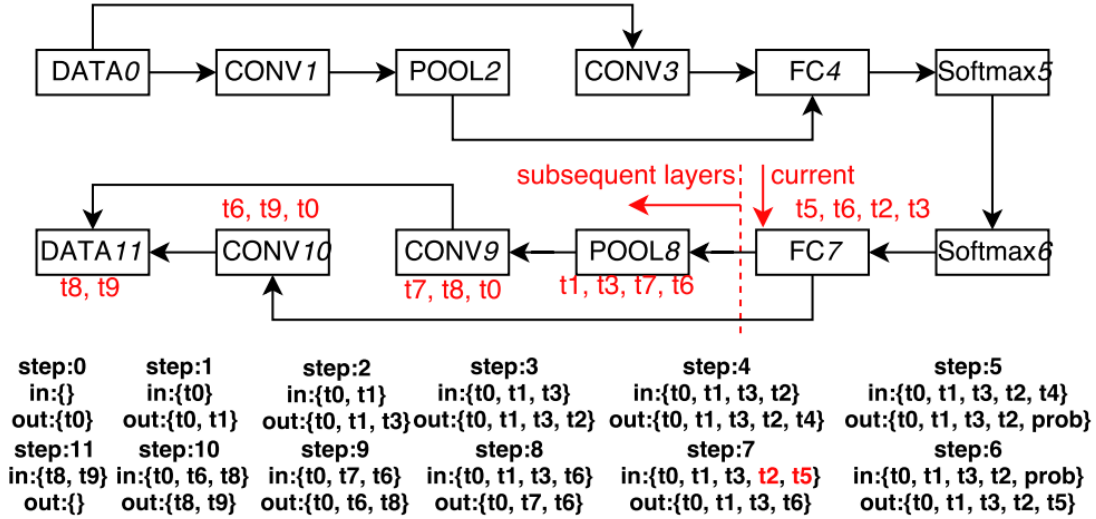


图 3.2 活跃度分析流程²

图 3.2 展示了活跃度分析的详细过程，图中列出了每个步骤的输入和输出集合中的数据。例如，对于 FC7，该层的输入集合中包括 t0, t1, t3, t2, t5 五个数据，当前层的计算产生新的中间结果 t6。但是由于 t2 和 t5 中不再参与所有后续层的计算，所以 FC7 的输出集合中直接删除 t2 和 t5，释放 t2 和 t5 所空余出的内存空间可以被其他中间结果重用。

活跃度分析将训练深度神经网络的内存开销 $peak_m = \sum_{i=1}^N l_i^f + \sum_{i=1}^N l_i^b$ 减少到 $\sum_{i=1}^N l_i^f + l_N^b$ 。为了简化分析，假设每一层的前向和后向计算所需要的内存空间相等，即 $l_i^f = l_i^b$ 。前向传播到第 k 层时，训练所产生的总的内存开销 $Cost_k^f = \sum_{i=1}^k l_i^f$ ($k \leq N$)。当反向传播到第 k 层时，活跃度分析就可以直接释放 l_i^f 和 l_i^b ($k \leq i \leq N$) 所对应的内存空间，所以反向传播到第 k 层时，训练所产生的总的内存开销为 $Cost_k^b = \sum_{i=1}^k l_i^f + l_k^b$ ($k \leq N$)。在整个训练过程中，内存开销的峰值为 $peak_m = \sum_{i=1}^N l_i^f + l_N^b$ ，相比传统的内存分配策略，活跃度分析能够节省接近 50% 的内存开销。

² SuperNeurons: Dynamic GPU Memory Management for Training Deep Neural Networks

3.1.2 交换数据到主存

深度神经网络通常由多个不同类型的层组成，而且神经网络通过随机梯度下降的方式进行训练。在多层迭代的训练中，每轮训练的计算过程都是固定的，按照层之间的先后顺序依次执行前向和后向计算。在前向传播中，某一层产生的特征图立刻被作为下一层的输入，参与下一层的前向计算，并在稍后的反向传播中参与计算。所以流行的深度学习框架采用面向整个网络的内存分配策略，将前向传播中产生的中间结果保存在 GPU 内存中，以便在反向传播中被使用。在给定的某个时刻，GPU 只能执行某一层的前向或后向计算，这将导致大量的中间结果在大部分的时间内都处于闲置状态。

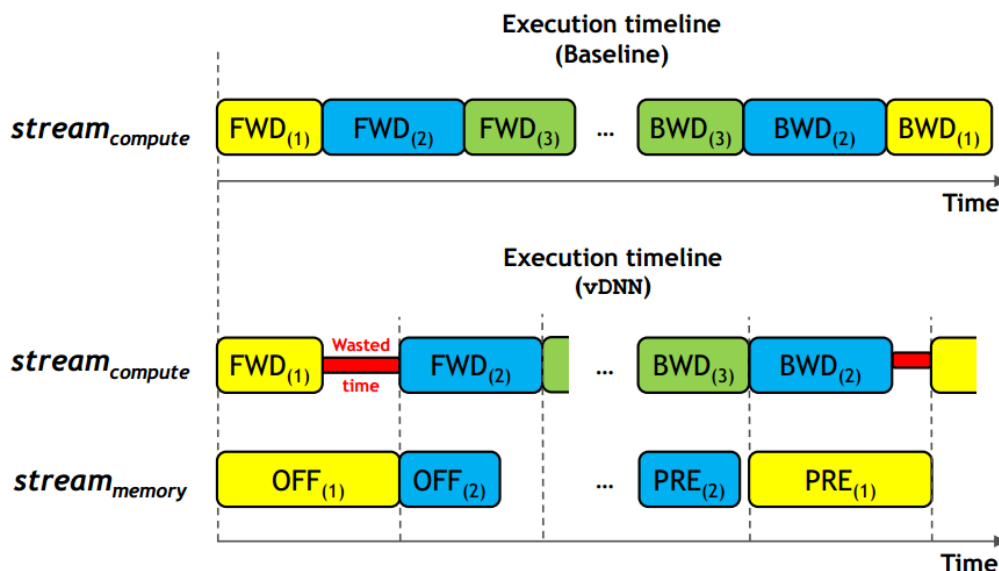


图 3.3 数据通信对性能的影响³

为了在有限的 GPU 内存中训练大规模的神经网络，可以将主存作为特征图的临时容器。基本的策略是在前向传播过程中，将生成的特征图从 GPU 内存卸载到主存中，当该特征图在反向传播中被重新使用时再从主存预取回 GPU 内存，但是将数据交换到主存中会引入大量的通信开销。由于 cuda^[31]中所有的操作都在流中执行，而且多个流中的计算和数据传输可以同时进行，所以部分通信开销可以被计算开销隐藏。如图 3.3 所示，当前层计算的同时，将输出特征图通过 PCIe 传输到主存中。由于特征图在前向传播中为只读的，所以此传输过程可以被计算操作安全的覆盖。只有在当前层的卸载操作完成后，才能开始进行下一层的计算操作。当被卸载的特征图的传

³ vDNN: Virtualized Deep Neural Networks for Scalable, Memory-Efficient Neural Network Design

输和依赖于该特征图的计算都完成后，GPU 内存中的该特征图的内存空间可以被释放。然而，当数据传输的开销比正向计算的开销更多时，交换数据到主存将会带来性能损失，影响训练速度。

为了在交换数据到主存的过程中尽可能的减少性能损失，并不需要对神经网络中的每一层都进行卸载和预取数据。通常越靠近输入层的特征图其被重用的时间距离就越远，在 GPU 内存中闲置的时间也越久，所以卸载和预取这些层的数据既能减少对性能的损失，又能减少大量的内存开销。在训练过程中，只需要选择某一层作为基准，比此层更靠近输入层的特征图都被交换到主存，其余层的特征图则直接保存在 GPU 内存中。通过将数据交换到主存，最低可使总内存开销等于网络中最大层的内存开销。

3.1.3 重计算

随着 GPU 的不断升级，其强大的并行计算能力可以极快的加快数据处理，然而 Pcie 的数据传输能力远远没有跟上 GPU 计算加快的速度。当网络模型不断变深变宽，交换数据到主存的方法会产生极大的数据传输开销，甚至远远超过计算开销，严重影响了深度神经网络的训练速度。本节介绍以计算换空间的重计算方法。在前向计算过程中，为了减少内存消耗，删除一些中间结果，这些数据在反向计算过程中被需要的时候借助额外的前向计算恢复他们。用来恢复这些数据的中间结果称为检查点。检查点将神经网络分成多个段，所以重计算的内存开销仅仅是所有检查点数据的大小和逐段反向传播时的最大内存占用。

在实践中，重计算方法大量节省内存，在整个训练过程中，内存开销的峰值降低为 $peak_m = \sum_{i=1}^N (l_i^f \in \text{checkpoints}) + l_N^b$ 。传统的重计算方法在选择检查点时通常只考虑生成检查点数据时的计算开销，所以通常将卷积层和工具层的前向输出结果作为检查点。但是直接将卷积层和工具层的输出保留在 GPU 内存中，忽略层之间在反向计算时的数据依赖，或者忽略非线性网络中不同非线性单元之间的数据依赖，都不能最大化的减少内存开销。

3.1.4 内存池

在训练过程中，需要经常通过 `cudaMalloc` 和 `cudaFree` 申请和释放中间结果的内存空间，而典型的训练阶段包括数百万次迭代，所以大量的内存操作会产生巨额的内存开销。根据实验，深度神经网络 ResNet50 使用 `cudaMalloc` 和 `cudaFree` 在内存操作

上浪费了 36.28% 的训练时间。GPU 内存池可以有效缓解内存操作带来的性能问题。内存池是一种内存抽象的内存分配方式，内存池^[32]技术在真正使用内存之前申请大块 GPU 内存作为共享内存池，这样做的一个显著优点是不必要每次都向系统申请内存，使得内存分配效率得到大幅度提升，除此之外，使用内存池还能有效解决内存碎片的问题，尤其是对训练深度神经网络这种内存密集型负载，解决内存碎片等同于增加可用内存空间。

3.2 基于重计算的 GPU 内存优化策略

3.2.1 面向非线性神经网络的 GPU 内存优化策略

由于在反向传播中借助重计算的方法恢复前向传播中丢弃的中间结果，所以首先需要确定神经网络中所有检查点的位置。检查点的设置方法如算法 3.1 所示，由于取消对 Concat 层的输出特征图的检查点设置，并且 Join 层的输出特征图重用临近的检查点的数据空间，所以非线性单元中，只有卷积层和 Norm 层输出可能作为检查点。利用此时的检查点集合确定 Join 层重用哪一个检查点的数据空间。然后为了最小化内存开销，将不参与反向计算的数据将不再作为检查点，可以在前向传播中直接丢弃该数据。所以如果 Join 层重用的是卷积层的输出特征图的数据空间，则该输出特征图在前向传播中直接被丢弃。通过如此的检查点设置，非线性单元中基本只保留 Norm 层的输出结果作为检查点，而且工具层在前向计算中不产生内存开销，即训练非线性神经网络时的内存消耗主要来自于 Norm 层。

算法 3.1 设置检查点

输入: *net*, 整个神经网络;

输出: *checkpoint*, 网络中的检查点

```
1  for layer l ∈ net do
2    如果该层是卷积层或者 Norm 层，则将其输出特征图作为检查点
3  end for
4  for c ∈ checkpoint do
5    通过活跃度分析，如果该检查点不参与任何反向计算，则取消该数据的检查点设置
6  end for
7  return checkpoint
```

确定检查点之后，在非线性网络的前向传播中，将检查点之外的中间结果直接丢弃。反向传播时再从最近的检查点通过重计算恢复这些中间结果。非线性网络中根据

卷积层、批规范化层和激活层的组合顺序不同,重计算的复杂度也不尽相同。在 BN-ACT-CONV 组合中,当把 Norm 层的输出特征图作为检查点,不同非线性单元之间的重计算过程相对独立,他们的重计算过程不存在数据依赖,重计算过程相对简单。而且需要重计算的层只有 Scale 和激活层,重计算的开销极小,却能在前向过程中释放除 Norm 层之外的所有输出特征图,带来大量的内存节省。在 CONV-BN-ACT 组合中,反向传播中重计算的过程相对复杂。如算法 3.2 所示,工具层的重计算过程相对复杂,尤其是 Noin 层的输出特征图重用 Norm 层检查点的数据空间时,重计算中需要引入 Join 层和 Scale 层的逆计算。但是 Join 层的逆计算和 Scale 层的逆计算的计算开销相对较小,而且面向非线性神经网络的内存优化策略能保证在前向过程中释放除 Norm 层之外的所有输出数据,减少内存占用。

算法 3.2 重计算

输入: 当前层序号, id; 检查点列表, P

输出: 当前层和其前继检查点之间的前向输出结果

```
1  给定当前层的序号 id, 获得当前层的前继检查点集合 pre_checkpoints
2  为 concat 层设置计数器, count = 0
3  for 前继检查点集合中的每一个检查点 do
4      当前重计算的层的序号 cur_l = pre_checkpointl.next_layer
5      if 当前层是 Join 层而且前继检查点是 Norm 层的前向输出结果 then
6          执行 Join 层的逆计算
7          执行 Scale 层的逆计算
8          continue
9      end if
10 while 当前重计算的层的序号 cur_l != id do
11     if 当前层是 Concat 层而且计数器 count 不等于 Concat 层输入的数量 then
12         count = count + 1
13         break
14     end if
15     当前层的前向计算
16     cur_l = cur_l.next_layer
17 end while
18 end for
19 return
```

面向非线性神经网络的内存优化策略通过调整检查点设置策略,综合利用空间复用和活跃度分析,降低需要常驻内存的检查点的总量,大大减少了训练非线性神经网络时的内存开销。

3.2.2 面向线性神经网络的 GPU 内存优化策略

非线性网络的瓶颈通常在某个比较大的卷积层，使得网络不能正常训练。在 MEDL 的实现中，系统通过静态分析每层计算所需要的内存大小，将瓶颈层的输入数据划分成多个 block，多个 block 的数据存储在连续的地址空间中，并重复进行相同的计算操作。但是在每个 block 的输入输出地址空间都不相同，通过便捷的地址偏移进行大量的空间复用。在前向和反向计算中，所有的 block 都共享同一个卷积层，共享同一组参数。在反向传播时，只需要对多个 block 的参数梯度进行累加，就能正确的进行模型训练。如算法 3.3 所示的瓶颈层的训练过程，每个 block 再进行前向或后向计算的同时，不断地在主存和 GPU 之间交换数据，最大程度的隐藏通信开销。算法中输出数据的累加表示在批次维度上的组合，并不是进行加法运算。

算法 3.3 瓶颈层的训练过程

输入：划分的 block 的数量，N

输出：瓶颈层的输出，Out

```
1  # 前向计算
2  for i = 0 ~ N do
3      Offload( block(i-1) )
4      Out += forward( blocki )
5  end for
6  # 反向计算
7  for i = N ~ 0 do
8      Prefetch( block(i-1) )
9      Out += backward( blocki )
10 end for
10 return out
```

为了保证多个 block 重复计算的同时进行数据传输，使用细粒度的双缓冲技术。其中缓冲区不是固定不变的，其随着 block 的移动而变化。细粒度的双缓冲使得不同 block 之间的数据空间可以重复利用，使得总体而言，瓶颈层在前向计算和反向计算中，输入和输出都是共享同一块内存空间，从而降低瓶颈层的内存开销。

3.3 本章小结

本章主要介绍了基于重计算的 GPU 内存管理器的实现，实现中用到的主要技术包括活跃度分析、重计算、交换数据到主存和内存池。通过活跃度分析和交换数据到主存引入更多的重计算，但是可以以微小的计算开销为代价减少大量的内存开销。

为了增强 MEDL 系统的可扩展性，MEDL 在重计算的基础上实现了面向非线性

网络的 GPU 内存优化策略和面向线性网络的 GPU 内存优化策略。对于非线性网络，通过活跃度分析调整网络中的检查点集合，降低需要常驻内存的检查点的总量，大大减少内存开销。检查点的调整为工具层引入更多的重计算，但是由于引入的重计算都是计算开销少的操作，并不会显著影响训练速度。对于线性网络，层内的重计算和细粒度的双缓冲技术为层内的空间重用提供可能，使得瓶颈层在前向和后向中，输入和输出数据都共享同一块数据空间。MEDL 使得更深更快的神经网络能在有限的 GPU 内存中进行训练。

4 系统测试与结果分析

本章对内存管理器进行测试并对实验结果进行分析。首先介绍系统 MEDL 的测试环境，测试方法和测试指标。然后从内存开销和性能两方面对比 SuperNeurons 系统和 InPlace-ABN 系统，并进行详细的测试结果分析。

4.1 测试环境

实验测试环境如表 4.1 所示，MEDL 基于 C++ 版本开发，使用的编译器版本为 4.6.4。训练过程在 GPU 设备上运行，需要安装显卡驱动和用于神经网络计算的 cudnn 库。系统均在 Linux 系统下测试。

表 4.1 实验环境配置

硬件配置	CPU	Intel(R) Xeon(R) CPU E5-2680 @ 2.40GHz (28 核)
	内存	256GB
	GPU	P100
	GPU 内存	16G
软件配置	操作系统	Ubuntu 16.04 LTS (x86_64)
	CUDA 驱动	cuda 8.0
	cudnn 版本	cudnn 7.1.3
	编译器	g++ 4.6.4

4.1.1 测试平台

对比实验中，MEDL 与 SuperNeurons 系统和在 SuperNeurons 中复现的 InPlace-ABN(IPABN)系统进行对比。SuperNeurons 系统的内存管理中包含活跃度分析、重计算，交换数据到主存、内存池等功能模块，能够显著的减少内存开销，但是其重计算设计缺乏对非线性网络的深入分析，检查点之间产生数据冗余。在对比实验中将会详细分析 MEDL 的对重计算的设计带来的内存节省。IPABN 提出 In-Place Activated BatchNorm layer 合并批规范化层和激活层的计算，然后将反向计算时批规范化层到激活层的重计算变成层内自包含的逆计算，从而可以轻易将 In-Place activated

batchnorm layer 整合到任何的深度学习框架中,即使该框架不支持重计算,但是其唯一的缺点是只能支持可逆的激活层,限制了 IPABN 的应用范围。

SuperNeurons 系统中每层的前向和后向计算都是调用 cudnn 的函数接口, cudnn 的实现中将批规范化的两个步骤归一化和尺度变换与偏移封装在一起并进行专门的性能优化,为了体现将批规范化层的中间数据作为检查点可以避免重计算过程中归一化的计算开销,而且为了方便重计算方法中检查点的设计,对比实验中的三个系统都使用单独实现的 Norm 层和 Scale 层的组合代替 cudnn 的批规范化实现。在复现 IPABN 时将 in-place activated batchnorm layer 替换成 Norm 层, Scale 层以及激活层的形式。SuperNeurons 中的检查点包括卷积层, Join 层, Concat 层的输出特征图, IPABN 中的检查点包括 Norm 层, Concat 层,以及 Join 层的输出特征图。系统 MEDL 中检查点只包括 Norm 层的输出特征图,其他特征图并不需要常驻内存。

4.1.2 测试用例和测试指标

训练深度神经网络需要大量的数据集,实验中采用标准的数据集 ImageNet 2012。ImageNet 数据集是为了促进图像识别技术的发展而创建的大型图像数据集,其中包含了大量分辨率高,种类丰富的图片,其含有的类别有上千个。深度学习系统的作用是训练深度神经网络,在实验中,用于测试的深度神经网络分别为 resNet50, pre-act resNet50, inception v4, denseNet121, vgg16 等,这些神经网络以数据集 ImageNet 2012 作为输入,在 GPU 中经过数以万计的迭代进行训练。由于 GPU 内存管理器的目标是减少内存开销,所以首要的测试指标为训练过程中 GPU 内存的使用量,其次在优化内存开销时,也要避免对训练速度造成影响,训练速度的度量指标为计算的吞吐量,即每秒训练的图片数量(img/s),每秒训练的图片数量越多,训练的速度就越快。

4.2 结果分析

4.2.1 非线性网络分析

系统 MEDL 与 SuperNeurons 和 IPABN 进行实验对比,从图 4.1 中可以看出, SuperNeurons 和 IPABN 在内存消耗上基本一致,而 MEDL 系统产生了大量的内存节省,这主要归功于对非线性单元的内存优化设计,避免了工具层在前向传播中对内存的消耗。图 4.1 中神经网络 resnet50 和 pre-act resnet50 均包含大量的 Join 层,但是 resnet50 能带来更多的内存节省(37.5%),这是因为神经网络 resnet50 的非线性单

元中卷积层,激活层和批规范化层(Norm+Scale)的先后顺序为 CONV-BN-ACT,其中 Norm 层的输出特征图和卷积层的输出特征图大小相同,而神经网络 pre-act resnet50 的非线性单元中采用 BN-ACT-CONV 的组合顺序,卷积层对输入特征图可能在通道维度或高和宽维度上进行升维或降维操作,所以 Norm 层的输出特征图大小和后续的卷积层的输出特征图大小存在差异。而且实际上神经网络 resnet50 和 pre-act resnet 50 相比, resnet50 除了 Norm 层的总输出小于 pre-act resnet50, 其他层的总输出大小相同。系统 MEDL 最终只有 Norm 层的输出常驻内存,所以 resnet50 具有比 pre-act resnet50 更高的内存节省(37.5% vs 22.8%)。图 4.1 中 inception v4 和 densenet121 中都包含大量 Concat 层,由于 densenet121 网络采用大量的短路连接以实现特征的重复利用,所以非线性单元中的 Concat 层不断地在通道维度组合数据,因此 Concat 层的输出特征图占用大量的内存。由于系统 MEDL 取消了将 Concat 层的输出特征图作为检查点,所以可以为 densenet 121 网络带来更多的内存节省(30.9%)。

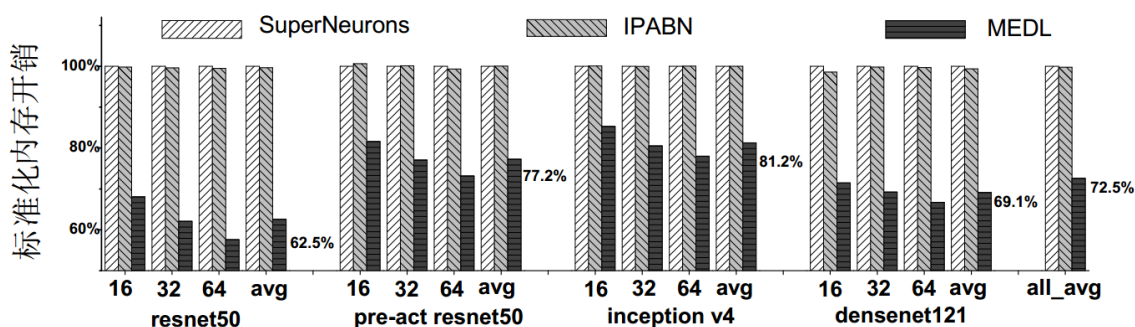


图 4.1 训练非线性网络时的内存占用

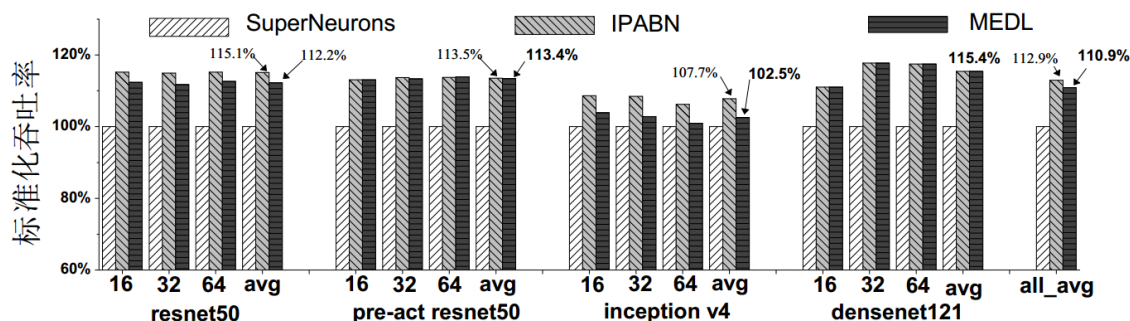


图 4.2 非线性网络的训练速度

如图 4.2 所示, 和 SuperNeurons 相比, IPABN 的训练速度在神经网络 resnet50, pre-act resnet50, inception v4 和 densenet121 中分别提升了 15%, 13.4%, 7.8%, 15.4%。

这是因为将 Norm 的输出特征图作为检查点时，可以在反向计算中减少归一化操作的计算开销，所以 IPABN 的训练速度更快。在非线性网络中，当卷积层，批规范化层，激活层的组合顺序为 CONV-BN-ACT 时，可以通过重计算的方式避免 Join/Concat 层的内存消耗，系统 MEDL 在反向计算引入更多的重计算开销，相比 IPABN 平均有 2% 的性能下降。但是由于 IPABN 只能使用可逆的激活函数，所以 IPABN 的使用具有局限性，而 MEDL 对所有的非线性网络都适用。如图 4.2 中的 pre-act resnet50 和 densenet121 所示，它们的卷积层，批规范化层，激活层的组合顺序为 CONV-BN-ACT。由于 Norm 层的反向计算不需要其输入特征图，所以此时邻近的非线性单元之间在反向计算过程中不存在数据依赖，也不需要通过重计算的方式恢复 Join 层或 Concat 层的输出特征图。此时在前向传播中直接丢弃 Join 层或 Concat 层的输出特征图，既能节省大量的内存，又不会产生额外的重计算开销。

综上所述，在非线性网络模型中，Norm 层的输出特征图的总量越少，MEDL 的内存节省的效果也越好。而且 MEDL 对层之间组合顺序为 BN-ACT-CONV 的非线性神经网络具有更好的性能提升，而组合顺序为 CONV-BN-ACT 的非线性单元由于会在反向传播时存在数据依赖，导致更多的重计算开销。

4.2.2 线性网络分析

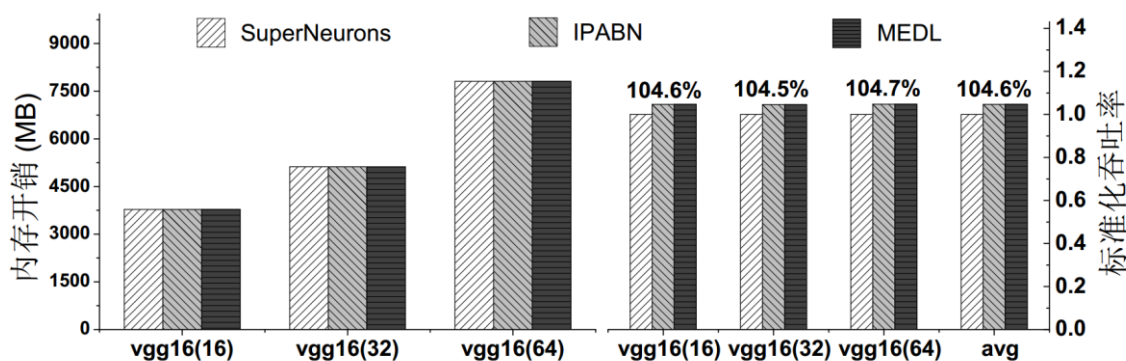


图 4.3 检查点设置对线性网络的影响

为了评估将 Norm 层的输出特征图设置为 checkpoint 时的训练加速情况，实验中使用带批规范化层的 vgg16 神经网络进行测试。因为神经网络 vgg16 中不包含非线性单元，所以系统 MEDL 和 SuperNeurons, IPABN 的区别主要在于 Norm 层和卷积层的不同检查点设置。在三个对比试验中，由于 Norm 层和 Scale 层的实现方式需要更频繁的为特征图申请和释放 GPU 内存，为了避免内存操作对训练速度的影响，所有的实验都采用内存池的方式统一申请 GPU 内存。如图 4.3 所示，而且系统 MEDL

和 IPABN 训练线性神经网络时加速效果相当, 平均提升 4.6%, 表明将 Norm 层的输出作为检查点确实可以加快训练速度, 因为在重计算过程中减少了归一化操作的计算开销。但是 IPABN 提出新的 in-place activated batchnorm layer 将批规范化层和激活层进行合并, 所以其有效性仅局限于使用可逆激活层的神经网络。

如图 4.3 所示, 三个系统在训练不同批次大小的 vgg16 网络时的内存消耗基本相同。即在前向传播中丢弃卷积层的输出特征图, 并将 Norm 层的输出特征图作为检查点常驻内存并没有减少总的内存开销。因为在带批规范化层的线性网络中 Norm 层和卷积层成对出现, 而且主要通过池化层进行特征图的下采样, 所以卷积层的输出特征图与 Norm 层的输出特征图在大小上没有区别。因此将 Norm 层的输出作为检查点可以减少重计算中归一化操作的计算开销, 但是对内存的优化基本没有作用。

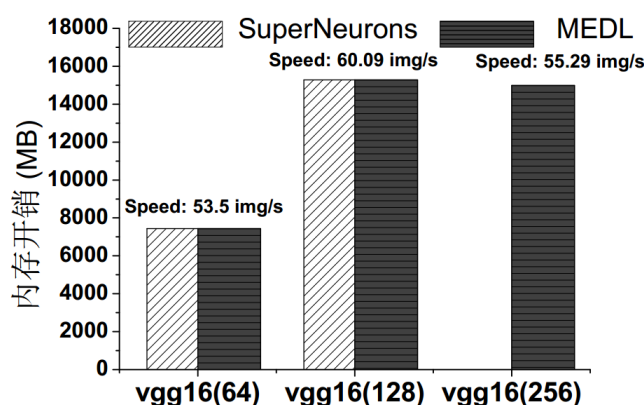


图 4.4 细粒度的双缓冲技术增加模型的扩展性

如图 4.4 所示, 仍然以线性网络中的 vgg16 为例进行实验, 测试细粒度的双缓冲技术对内存开销的影响。vgg16 在线性网络中已经算是比较深的网络, 而且训练 vgg16 网络所需要的内存也相对较大。在实验中, 对 MEDL 和 SuperNeurons 都启动 LRU(交换数据到主存)和重计算模块。当 GPU 内存不足时, 将当前层计算不需要的数据卸载到主存中。批次大小小于 128 的 vgg16 网络在 SuperNeurons 上都能正常训练, 但是当批次大小等于 256 时, vgg16 网络中最大层的数据大小为 12G, 加上工作区的内存消耗和系统运行时的内存消耗, 导致拥有 16G 内存的 GPU P100 不能正常进行训练。如图 4.4 所示, 批次大小等于 256 时, 系统 SuperNeurons 不能进行训练, 而系统 MEDL 还能继续进行训练, 正是由于使用细粒度的双缓冲技术减少了瓶颈层的内存开销。但是由于部分通信开销不能被计算隐藏, 导致使用细粒度双缓冲时训练速度有所下降。所以为了在有限的 GPU 内存中训练更宽的线性网络, 必须以训练速度为代价。

4.3 本章小结

本章对 GPU 内存管理方法(MEDL)进行了全面的测试并分别探究了该方法对非线性神经网络和线性神经网络的影响。首先介绍了实验环境并介绍了数据集和实验中测试的各种深度神经网络,接下来介绍了训练非线性神经网络时,内存管理方法对内存开销和训练速度的影响,说明了 MEDL 相比传统方法的优越性。最后通过线性网络的实验对比分析说明了 MEDL 能够有效解决瓶颈层的内存开销问题。总体而言, MEDL 具有更好的模型扩展性,在有限的 GPU 内存中可以训练更深更宽的神经网络。

5 总结与展望

5.1 工作总结

随着深度神经网络的不断变深变宽，训练神经网络时的计算量和训练过程中产生的中间结果都越来越大，对训练设备的计算能力和内存容量也提出了更高的要求。由于 GPU 强大的计算能力，其已经成为训练神经网络模型的首选设备。但是 GPU 的内存容量相对不足。如何在有限的 GPU 内存中训练更深更宽的神经网络，给深度学习系统的设计带来了巨大的挑战。

为了训练更深更宽的神经网络，本文提出了基于重计算的内存优化方法，通过引入更多的重计算，大大减少了训练神经网络时的内存开销。实验结果表明，在有限的 GPU 内存中，系统 MEDL 可以训练更深更宽的神经网络，而引入的重计算只带来少量的计算开销。

本论文的主要内容总结如下：

1. 通过对反向计算中数据依赖的深入分析，调整检查点的设置策略，减少了重计算中归一化操作的计算开销。
2. 设计并实现了面向非线性神经网络的 GPU 内存优化策略。非线性神经网络中存在大量的工具层，传统的重计算方法为了反向计算的顺利进行，将工具层的输出特征图作为检查点常驻内存。系统 MEDL 通过空间重用和活跃度分析，使工具层在前向传播过程中不增加内存开销，降低训练过程中总的内存开销，使得更深的神经网络能够在有限的 GPU 内存中进行训练。
3. 设计并实现了面向线性神经网络的 GPU 内存优化策略。对于线性网络中的瓶颈层，本文使用更细粒度的双缓冲技术，支持层内重计算，减少瓶颈层在前向和后向传播中的内存开销，使得更宽的神经网络能够在有限的 GPU 内存中进行训练。

5.2 未来展望

虽然本文针对训练深度神经网络时存在的 GPU 内存不足问题提出了相应的解决方案，但 MEDL 系统仍然存在一些待改进的地方：

1. 面向数据流图的计算优化：本文中提出的内存管理的实现是基于 SuperNeurons 系统，SuperNeurons 系统必须是一层接一层的计算方式，层之间的计算

不会产生重叠。而在 TensorFlow 或 PyTorch 等深度学习系统中，其首先通过构建数据流图^[33]，再对数据流图进行计算合并^[34]和空间复用，使训练中的计算更高效，更节省内存。尤其是对于非线性网络中存在多个分支的情况，很多层的计算完全可以并行处理。如果可以在 SuperNeurons 中实现数据流图，并面向数据流图进行计算和内存优化，那么就能更好的降低训练神经网络时的内存开销，以及加快训练速度。

2. 构建分布式的深度学习系统^[35-37]：深度学习训练非常耗时，所以一般采用多台设备通过数据并行的方式进行训练，也是训练大规模深度神经网络的最直接的方式。数据并行的方式可以降低对单块 GPU 的内存需要，但是多台机器并行训练时需要不断同步参数^[38, 39]。训练过程需要经过数以百万计的迭代，所以参数同步会产生大量且频繁的通信开销。为了避免参数更新影响训练速度，可以使用 RDMA^[40, 41]进行设备间的数据传输，大大降低参数同步的开销。若有效的使用 RDMA 和 MEDL 实现分布式的深度学习系统，则可以通过数据并行的方式，在有限的 GPU 内存中，训练更深更宽的深度神经网络。

致 谢

转眼间研究生生涯已经快要结束！回想这三年来的一点一滴，一切恍如昨日。三年前，一个对科研毫无经验的我怀着对计算机体系结构，深度学习等技术的向往加入CGCL大家庭。一切从零开始对我来说并不容易，幸运的是能够得到实验室各位老师的指导。这三年中，我学会了如何调研课题，扩展思路，也通过项目实践提升了我对计算机体系结构和深度学习系统认识和理解。在这里，我要感谢实验室所有老师、同学以及我的家人和朋友，是你们的帮助和关心让我逐渐成长起来。在本论文即将完成之际，我要向你们送上最诚挚的感谢和祝福。

首先，我要感谢我的导师廖小飞老师。廖老师学识渊博，精通计算机系统结构、分布式和虚拟化等领域的前沿知识，能够精准地把握学科前沿。生活中，廖老师待人亲和，时刻为学生着想；学习和科研工作上，廖老师不断给予帮助，关心每一个学生的科研进度，在研究生第一年我一直找准科研方向时，正是廖老师分享的一篇文章为我打开了思路，并且顺利完成课题研究。感谢廖老师一直以来对我的鼓励和帮助。

感谢美国密歇根理工大学的岳建辉老师，岳老师教会了我严谨认真的科研态度，在两年半的科研时光中，有您的指导是我格外的幸运。在学术上，岳老师恪尽职守、兢兢业业，为了扩展我的科研思路，经常和我一起研读论文，一起讨论，把科研变成一件有乐趣的事情。在我的研究成果和别人撞车，导致论文不能发表时，感谢您对我的鼓励和帮助。毕业在即，衷心感谢您对我的不断指导。

感谢实验室主任金海老师，金老师科研成果丰硕，治学态度严谨，时刻把握着国内外研究热点，是不断推进实验室发展的领头人。为了拓展学生的学术视野，金老师经常邀请国内外专家到实验室作学术讲座；为了促进学生和老师之间的交流、总结规划实验室年度工作，实验室每年都会举行暑期年会。

感谢实验室的郑龙老师。在学术上，郑老师严谨认真，对操作系统、编译和图计算领域都有深刻的理解，郑老师每周都组织小组讨论会讨论最新的研究进展，您严谨治学、实事求是的科研态度为我们树立了很好的榜样。感谢您对我科研道路上的无私帮助和大量付出。

感谢实验室的每位老师，感谢刘英书老师为我们营造了一个舒适的学习和科研环境，感谢王多强和章勤老师耐心地安排毕业答辩工作，同样感谢陆枫老师、郑然老师、吴未老师、刘方明老师、吴松老师等等，感谢他们对实验室的付出。

华中科技大学硕士学位论文

感谢已经毕业的郭人通师兄，他是一个科研科研牛人，也是一个工作狂人。感谢他带我走进深度学习系统的大门，在科研的最初阶段，带我领略了系统软件领域的魅力。在我科研遇到困难时，是你为我提供了方向。感谢陈湃学长在我科研过程中为我提供的建议，以及在找工作过程中为我提供的帮助。感谢李贤亮同学、叶源远同学和杨荡同学，在我科研遇到困难时及时的帮助，在我撰写论文时提供的修改意见。感谢我的大学同学张玮玮和胡蝶同学，感谢你们从大学来一直的陪伴，你们是我的好朋友，也是我学习的榜样，还要感谢我研究生期间的室友林松、张振、李翌等同学，你们的广博的学识、积极的生活态度让我受益匪浅，感谢你们的陪伴，祝愿你们工作顺利，未来生活一帆风顺。

最后感谢我的奶奶、父母、弟弟和我的女朋友，你们支持和鼓励是我不断努力的动力，谢谢你们一直以来对我的包容、关心和照顾。

最后再次对所有曾经帮助过和关心我的人表示衷心的感谢。

参考文献

- [1] Krizhevsky A, Sutskever I, Hinton G E. Imagenet classification with deep convolutional neural networks. Advances in neural information processing systems. 2012: 1097-1105.
- [2] Simonyan K, Zisserman A. Very deep convolutional networks for large-scale image recognition. arXiv preprint:1409.1556, 2014.
- [3] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition, Proceedings of the IEEE conference on computer vision and pattern recognition. 2016: 770-778.
- [4] Szegedy C, Liu W, Jia Y, et al. Going deeper with convolutions. Proceedings of the IEEE conference on computer vision and pattern recognition. 2015: 1-9.
- [5] Huang G, Liu Z, Van Der Maaten L, et al. Densely connected convolutional networks. Proceedings of the IEEE conference on computer vision and pattern recognition. 2017: 4700-4708.
- [6] Ioffe S, Szegedy C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. arXiv preprint:1502.03167, 2015.
- [7] Szegedy C, Vanhoucke V, Ioffe S, et al. Rethinking the inception architecture for computer vision. Proceedings of the IEEE conference on computer vision and pattern recognition. 2016: 2818-2826.
- [8] Szegedy C, Ioffe S, Vanhoucke V, et al. Inception-v4, inception-resnet and the impact of residual connections on learning. Thirty-First AAAI Conference on Artificial Intelligence. 2017.
- [9] Dean J, Corrado G, Monga R, et al. Large scale distributed deep networks. Advances in neural information processing systems. 2012: 1223-1231.
- [10] Jia Y, Shelhamer E, Donahue J, et al. Caffe: Convolutional architecture for fast feature embedding. Proceedings of the 22nd ACM international conference on Multimedia. ACM, 2014: 675-678.
- [11] Collobert R, Bengio S, Mariéthoz J. Torch: a modular machine learning software library. Idiap, 2002.
- [12] Chen T, Li M, Li Y, et al. Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems. arXiv preprint:1512.01274, 2015.

- [13] Abadi M, Barham P, Chen J, et al. Tensorflow: A system for large-scale machine learning. 12th USENIX Symposium on Operating Systems Design and Implementation. 2016: 265-283.
- [14] Rhu M, Gimelshein N, Clemons J, et al. vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design. The 49th Annual IEEE/ACM International Symposium on Microarchitecture. IEEE Press, 2016: 18.
- [15] Cui H, Zhang H, Ganger G R, et al. Geeps: Scalable deep learning on distributed gpus with a gpu-specialized parameter server. Proceedings of the Eleventh European Conference on Computer Systems. ACM, 2016: 4.
- [16] Chen T, Xu B, Zhang C, et al. Training deep nets with sublinear memory cost. arXiv preprint:1604.06174, 2016.
- [17] Wang L, Ye J, Zhao Y, et al. Superneurons: dynamic gpu memory management for training deep neural networks. ACM SIGPLAN Notices. ACM, 2018, 53(1): 41-53.
- [18] Rota Bulò S, Porzi L, Kontschieder P. In-place activated batchnorm for memory-optimized training of dnns. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2018: 5639-5647.
- [19] Markthub P, Belviranli M E, Lee S, et al. DRAGON: breaking GPU memory capacity limits with direct NVM access. Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis. IEEE Press, 2018: 32.
- [20] Chen T, Du Z, Sun N, et al. Diannao: A small-footprint high-throughput accelerator for ubiquitous machine-learning. ACM Sigplan Notices. ACM, 2014, 49(4): 269-284.
- [21] Chen Y, Luo T, Liu S, et al. Dadiannao: A machine-learning supercomputer. Proceedings of the 47th Annual IEEE/ACM International Symposium on Microarchitecture. IEEE Computer Society, 2014: 609-622.
- [22] Chen Y H, Krishna T, Emer J S, et al. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. IEEE Journal of Solid-State Circuits, 2017, 52(1): 127-138.
- [23] Chen Y H, Emer J, Sze V. Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks. ACM SIGARCH Computer Architecture News. IEEE Press, 2016, 44(3): 367-379.
- [24] LiKamWa R, Hou Y, Gao J, et al. RedEye: analog ConvNet image sensor architecture for continuous mobile vision. ACM SIGARCH Computer Architecture News. IEEE Press, 2016, 44(3): 255-266.

- [25] Reagen B, Whatmough P, Adolf R, et al. Minerva: Enabling low-power, highly-accurate deep neural network accelerators. 2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture. IEEE, 2016: 267-278.
- [26] Han S, Liu X, Mao H, et al. EIE: efficient inference engine on compressed deep neural network. 2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture. IEEE, 2016: 243-254.
- [27] Hassibi B, Stork D G. Second order derivatives for network pruning: Optimal brain surgeon. Advances in neural information processing systems. 1993: 164-171.
- [28] 刘娜, 李翠华. 基于多层卷积神经网络学习的单帧图像超分辨率重建方法. 中国科技论文, 2015, 10(2):201-206.
- [29] Vanhoucke V, Senior A, Mao M, et al. Improving the speed of neural networks on cpus. In Proc. Deep Learning and Unsupervised Feature Learning NIPS Workshop. 2011: 4-5.
- [30] Judd P, Albericio J, Hetherington T, et al. Proteus: Exploiting numerical precision variability in deep neural networks. Proceedings of the 2016 International Conference on Supercomputing. ACM, 2016: 23.
- [31] ShaneCook, 库克, 苏统华. CUDA 并程序序设计:GPU 编程指南. 机械工业出版社, 2014.
- [32] 郭丙轩, 张京莉, 张志超. 基于内存池的空间数据调度算法. 计算机工程, 2008, 34(6).
- [33] Ghamarian A H, Geilen M C W, Stuijk S, et al. Throughput analysis of synchronous data flow graphs. Sixth International Conference on Application of Concurrency to System Design. IEEE, 2006: 25-36.
- [34] Wongsuphasawat K, Smilkov D, Wexler J, et al. Visualizing dataflow graphs of deep learning models in tensorflow. IEEE transactions on visualization and computer graphics, 2018, 24(1): 1-12.
- [35] Ho Q, Cipar J, Cui H, et al. More effective distributed ml via a stale synchronous parallel parameter server. Advances in neural information processing systems. 2013: 1223-1231.
- [36] Zhang H, Zheng Z, Xu S, et al. Poseidon: An Efficient Communication Architecture for Distributed Deep Learning on GPU Clusters. 2017 USENIX Annual Technical Conference. 2017: 181-193.

- [37] Kim J K, Ho Q, Lee S, et al. STRADS: a distributed framework for scheduled model parallel machine learning. Proceedings of the Eleventh European Conference on Computer Systems. ACM, 2016: 5.
- [38] Cui H, Tumanov A, Wei J, et al. Exploiting iterative-ness for parallel ML computations. Proceedings of the ACM Symposium on Cloud Computing. ACM, 2014: 1-14.
- [39] Wei J, Dai W, Qiao A, et al. Managed communication and consistency for fast data-parallel iterative analytics. Proceedings of the Sixth ACM Symposium on Cloud Computing. ACM, 2015: 381-394.
- [40] Liu J, Wu J, Panda D K. High performance RDMA-based MPI implementation over InfiniBand. International Journal of Parallel Programming, 2004, 32(3): 167-198.
- [41] Kalia A, Kaminsky M, Andersen D G. Using RDMA efficiently for key-value services. ACM SIGCOMM Computer Communication Review, 2015, 44(4): 295-306.

附录 1 攻读学位期间被录用的会议论文

- [1] Long Zheng, Xianliang Li, Xi Ge, Xiaofei Liao, Zhiyuan Shao, Hai Jin, Qiang-Sheng Hua. Efficient Graph Processing with Invalid Update Filtration. (Accepted by IEEE Transactions on Big Data)
- [2] Xi Ge, Jianhui Yue, Xiaofei Liao, Hai Jin, Xianliang Li. MEDL: Memory Efficient Deep Learning Training on GPU. (In the process of contributions)

附录 2 攻读学位期间申请的国家发明专利

- [1] 金海, 廖小飞, 郑龙, 刘海坤, 葛希. 一种基于层宽内存分配的深度学习异构计算方法和系统. 专利申请号: 201910136545.8